

Programmation JAVA : 3 – L2 UPEC

TP 2 : Collections, comparaisons et généricité

Année universitaire 2026–2027 — 25 septembre 2026

Contexte et objectifs

Dans ce TP, vous allez gérer une collection de produits, puis un entrepôt de boîtes génériques. Vous apprendrez à :

- définir un ordre naturel avec `Comparable<T>` ;
- créer et utiliser des comparateurs avec `Comparator<T>` ;
- écrire des méthodes et des classes génériques ;
- représenter l'absence de résultat avec `Optional<T>` ;
- regrouper des objets à l'aide d'une `Map<K, V>`.

Organisation du travail dans VS Code

Utilisez **Java 25** ou une version ultérieure et VS Code avec l'extension **Extension Pack for Java** à jour.

1. Créez un dossier **TP2-Magasin**, puis ouvrez ce dossier dans VS Code avec **Fichier > Ouvrir un dossier...**
2. Dans l'explorateur de VS Code, créez vos fichiers Java directement dans ce dossier, au fur et à mesure des questions : `Produit.java`, `Magasin.java`, `Utils.java`, etc. Chaque classe est écrite dans le fichier portant son nom.
3. Créez également `EssaiMagasin.java`, qui contiendra le petit programme principal ci-dessous. Complétez-le au fil des étapes pour tester votre travail.
4. Enregistrez les fichiers, puis cliquez sur **Run** au-dessus de `void main()`. VS Code compile les fichiers nécessaires puis lance le programme. Le résultat apparaît dans le terminal intégré.

Premier essai dans `EssaiMagasin.java` :

```
1 void main() {  
2     IO.print("Bienvenue dans ");  
3     IO.println("le magasin !");  
4 }
```

Sortie attendue :

```
Bienvenue dans le magasin !
```

Les programmes principaux utilisent un `main` compact. Les classes comme `Produit` et `Magasin` restent des classes déclarées explicitement. Dans ces classes, ajoutez les instructions `import` nécessaires en haut du fichier. Par exemple, pour utiliser `List`, écrivez `import java.util.List;`. Les extraits sans `main` sont à insérer dans le programme principal.

Pour l'exercice 2 : créez un autre dossier, **TP2-Entrepot**, et ouvrez-le dans une nouvelle fenêtre de VS Code. Créez-y directement `Boite.java`, `Entrepot.java` et un petit programme `EssaiEntrepot.java` pour tester progressivement votre travail. Chaque fenêtre doit être ouverte sur le dossier de l'exercice concerné.

Si Run n'apparaît pas : attendez la fin du chargement des outils Java et vérifiez les erreurs signalées dans l'onglet **Problèmes**. Pour vérifier le JDK utilisé, ouvrez la palette de commandes et recherchez **Java: Configure Java Runtime** : choisissez un JDK 25 ou ultérieur pour le dossier courant.

Programmes principaux fournis

Deux fichiers accompagnent l'énoncé : `TestMagasin.java` et `TestEntrepot.java`. Ils reprennent les programmes complets présentés dans les exercices.

Copiez chaque fichier dans le dossier de son exercice uniquement à l'étape « Programme principal ». Avant cette étape, utilisez votre fichier `EssaiMagasin.java` ou `EssaiEntrepot.java` pour effectuer les essais proposés. Les programmes fournis utilisent des classes et des méthodes que vous devez d'abord écrire.

À l'étape consacrée à **Optional**, vous adapterez vous-même `TestMagasin.java` au nouveau type de résultat.

Exercice 1 : Gestion de produits dans un magasin

Étape 1 : Classe Produit avec Comparable

Créer une classe `Produit` représentant un article en stock.

Cahier des charges :

- La classe implémente `Comparable<Produit>`.
- Elle possède trois attributs privés et `final` : `nom (String)`, `prix (double)` et `stock (int)`.
- Elle fournit un constructeur complet, des getters et une méthode `toString()`.
- Le constructeur refuse un nom nul ou vide, un prix négatif ou non fini et un stock négatif. Lancez une `IllegalArgumentException` avec un message explicite. Pour le prix, vous pouvez utiliser `Double.isFinite(prix)`. Aucun `try/catch` n'est demandé.
- La méthode `compareTo(Produit autre)` définit l'ordre naturel par prix croissant, avec `Double.compare(this.prix, autre.prix)`.

Exemple d'utilisation :

```
1 var p1 = new Produit("Pommes", 2.5, 10);
2 var p2 = new Produit("Bananes", 1.8, 20);
3 IO.println(p1);
4 IO.println(p1.compareTo(p2));
```

Sortie attendue :

```
Pommes (2.5 € - stock : 10)
1
```

Le signe du résultat indique l'ordre : négatif, nul ou positif. Un résultat nul signifie ici que les deux produits ont le même prix, pas nécessairement le même nom ou le même stock.

Étape 2 : Classe Magasin

Créer une classe `Magasin` qui contient une collection de produits.

Cahier des charges :

- Un attribut privé `final` nommé `produits`, de type `List<Produit>`.
- Un constructeur sans argument initialisant une `ArrayList` vide.
- Une méthode `ajouterProduit(Produit produit)` ; un produit nul est refusé avec une `IllegalArgumentException`.
- Une méthode `getProduits()` retournant une copie non modifiable de la liste, obtenue avec `List.copyOf(produits)`.
- Une méthode `toString()` donnant la représentation de la liste des produits.

La copie retournée protège la liste interne du magasin. Les objets `Produit` sont partagés, mais leurs attributs ne sont pas modifiables dans ce TP. L'attribut `final` interdit de réaffecter la variable `produits`, pas d'ajouter des éléments à la liste.

Exemple d'utilisation :

```

1 var magasin = new Magasin();
2 magasin.ajouterProduit(new Produit("Ordinateur", 1200.50, 10));
3 magasin.ajouterProduit(new Produit("Smartphone", 800.00, 25));
4 IO.println(magasin);

```

Sortie attendue (présentée sur deux lignes pour la lisibilité) :

```

[Ordinateur (1200.5 € - stock : 10),
 Smartphone (800.0 € - stock : 25)]

```

Étape 3 : Classe Utils — méthodes génériques max et min

Créer une classe `Utils` contenant deux méthodes statiques et génériques `max` et `min`. Elles reçoivent une `List<T>` et retournent respectivement son plus grand et son plus petit élément selon l'ordre naturel.

Contraintes et indications :

- Le type `T` doit permettre de comparer les éléments entre eux. Utilisez la borne `T extends Comparable<T>`.
- Parcourez la liste avec une boucle et utilisez `compareTo`. Ne trie pas et ne modifiez pas la liste.
- Si la liste est vide, retournez `null` à cette étape.
- Si la liste elle-même est `null`, lancez une `IllegalArgumentException`. On suppose que les éléments de la liste ne sont pas nuls.
- En cas d'égalité selon l'ordre naturel, conservez le premier élément rencontré.

Exemple d'utilisation :

```

1 var produits = List.of(
2     new Produit("Ordinateur", 1200.50, 10),
3     new Produit("Smartphone", 800.00, 25),
4     new Produit("Tablette", 600.00, 15)
5 );
6 IO.println("Produit le plus cher : " + Utils.max(produits));
7 IO.println("Produit le moins cher : " + Utils.min(produits));

```

Sortie attendue :

```

Produit le plus cher : Ordinateur (1200.5 € - stock : 10)
Produit le moins cher : Tablette (600.0 € - stock : 15)

```

Vérifiez ensuite que les **mêmes méthodes**, sans modification, fonctionnent aussi avec des entiers et des chaînes :

```

1 IO.println(Utils.max(List.of(4, 9, 2)));
2 IO.println(Utils.min(List.of("poire", "abricot", "pomme")));
3 List<Produit> vide = List.of();
4 IO.println(Utils.max(vide));

```

Sortie attendue :

```

9
abricot
null

```

Testez également une liste d'un seul élément et une liste contenant deux produits de même prix.

Étape 4 : Tri avec des comparateurs nommés

Créer trois classes implémentant `Comparator<Produit>` :

- `CompareurNom` : ordre alphabétique des noms avec `String.compareTo` ;
- `CompareurPrix` : prix croissants ;
- `CompareurStock` : stocks décroissants, avec `Integer.compare` en inversant l'ordre des arguments.

L'interface `Comparator<T>` définit la méthode `int compare(T o1, T o2)`. Son résultat est négatif si `o1` vient avant `o2`, nul s'ils sont équivalents selon le critère choisi, et positif si `o1` vient après `o2`.

Exemple de classe :

```

1 import java.util.Comparator;
2
3 public class CompareurPrix implements Comparator<Produit> {
4     @Override
5     public int compare(Produit p1, Produit p2) {
6         return Double.compare(p1.getPrix(), p2.getPrix());
7     }
8 }

```

La méthode `sort` de l'interface `List` trie une liste modifiable à l'aide d'un comparateur :

```

1 var produits = new ArrayList<Produit>();
2 produits.add(new Produit("Pommes", 2.5, 10));
3 produits.add(new Produit("Bananes", 1.8, 20));
4 produits.sort(new CompareurNom());
5 IO.println(produits);

```

Sortie attendue :

```
[Bananes (1.8 € - stock : 20), Pommes (2.5 € - stock : 10)]
```

Ajouter dans `Magasin` trois méthodes : `trierParNom()`, `trierParPrix()` et `trierParStock()`. Elles utilisent ces comparateurs pour trier **la liste interne**. Il ne faut pas trier le résultat non modifiable de `getProduits()`.

Étape 5 : Programme principal

Copiez le fichier fourni `TestMagasin.java` dans le dossier `TP2-Magasin`. Il contient le programme ci-dessous. Enregistrez tous vos fichiers, puis lancez-le avec **Run** au-dessus de `main`. Comparez son affichage avec la sortie attendue et corrigez vos classes si nécessaire.

```

1 void main() {
2     var m = new Magasin();
3     m.ajouterProduit(new Produit("Pommes", 2.5, 10));
4     m.ajouterProduit(new Produit("Bananes", 1.8, 20));
5     m.ajouterProduit(new Produit("Pêches", 3.2, 15));
6
7     IO.println("=== Liste initiale ===");
8     IO.println(m);
9
10    m.trierParNom();
11    IO.println("=== Tri par nom ===");
12    IO.println(m);
13
14    m.trierParPrix();
15    IO.println("=== Tri par prix ===");
16    IO.println(m);
17
18    m.trierParStock();
19    IO.println("=== Tri par stock (décroissant) ===");
20    IO.println(m);
21
22    IO.println("=== Produit le plus cher ===");
23    IO.println(Utils.max(m.getProduits()));
24    IO.println("=== Produit le moins cher ===");
25    IO.println(Utils.min(m.getProduits()));
26 }

```

Sortie attendue (les listes sont présentées sur plusieurs lignes) :

```

=== Liste initiale ===
[Pommes (2.5 € - stock : 10), Bananes (1.8 € - stock : 20),
 Pêches (3.2 € - stock : 15)]
=== Tri par nom ===
[Bananes (1.8 € - stock : 20), Pommes (2.5 € - stock : 10),
 Pêches (3.2 € - stock : 15)]
=== Tri par prix ===
[Bananes (1.8 € - stock : 20), Pommes (2.5 € - stock : 10),
 Pêches (3.2 € - stock : 15)]
=== Tri par stock (décroissant) ===
[Bananes (1.8 € - stock : 20), Pêches (3.2 € - stock : 15),
 Pommes (2.5 € - stock : 10)]
=== Produit le plus cher ===
Pêches (3.2 € - stock : 15)
=== Produit le moins cher ===
Bananes (1.8 € - stock : 20)

```

Étape 6 : Représenter l'absence de résultat avec Optional

Modifier max et min pour qu'elles retournent un `Optional<T>` :

- si la liste est vide, retourner `Optional.empty()` ;
- sinon, retourner `Optional.of(resultat)`, où `resultat` est l'élément trouvé ;
- une liste null est refusée avec une `IllegalArgumentException`.

Il ne faut pas créer un produit fictif pour représenter l'absence de résultat. Un `Optional` est soit vide, soit porteur d'une valeur.

Exemple d'utilisation :

```

1 List<Produit> vide = List.of();
2 Optional<Produit> maxVide = Utils.max(vide);
3 IO.println(maxVide.isEmpty());
4
5 var produits = List.of(new Produit("Pommes", 2.5, 10));
6 Optional<Produit> maximum = Utils.max(produits);
7 if (maximum.isPresent()) {
8     IO.println(maximum.get());
9 } else {
10     IO.println("Aucun produit.");
11 }
    
```

Sortie attendue :

```

true
Pommes (2.5 € - stock : 10)
    
```

Ici, `get()` n'est appelé qu'après avoir vérifié la présence d'une valeur. Modifiez le fichier fourni `TestMagasin.java` pour conserver son affichage et afficher "Aucun produit." lorsque le magasin est vide. Adaptez aussi les tests sur les entiers et les chaînes.

Exercice 2 : Gestion d'un entrepôt de boîtes génériques

Cet exercice prolonge le premier. Il est à commencer en séance si le temps le permet et à poursuivre à la maison. Ouvrez le dossier `TP2-Entrepot` dans une nouvelle fenêtre de VS Code, comme indiqué au début du TP.

Étape 1 : Classe générique Boite<T>

Créer une classe `Boite<T>` représentant une boîte contenant un objet de type `T`.

Cahier des charges :

- La classe implémente `Comparable<Boite<T>>`.
- Elle possède deux attributs privés et `final` : `contenu (T)` et `poids (double)`.
- Elle fournit un constructeur complet, des getters et `toString()`.
- Le constructeur refuse un contenu nul et un poids négatif ou non fini, avec une `IllegalArgumentException`.
- La méthode `compareTo(Boite<T> autre)` compare les poids par ordre croissant, avec `Double.compare`.

Il n'est pas nécessaire que `T` implémente `Comparable` : on compare les **poids**, pas les contenus.

Exemple d'utilisation :

```

1 Boite<String> b1 = new Boite<>("Pommes", 2.5);
2 Boite<String> b2 = new Boite<>("Pommes", 1.8);
3 IO.println(b1);
4 IO.println(b1.compareTo(b2));
5 Boite<Integer> b3 = new Boite<>(42, 0.5);
6 IO.println(b3.getContenu());
    
```

Sortie attendue :

```

Boite{contenu=Pommes, poids=2.5}
1
42
    
```

Étape 2 : Classe générique Entrepot<T>

Créer une classe `Entrepot<T>` contenant des boîtes dont les contenus ont le même type `T`.

Cahier des charges :

- Un attribut privé `final` nommé `boites`, de type `List<Boite<T>>`.
- Un constructeur sans argument initialisant une liste modifiable vide.
- Une méthode `ajouterBoite(Boite<T> boite)` ; une boîte nulle provoque une `IllegalArgumentException`.
- Une méthode `toString()` retournant la représentation de la liste.

Exemple d'utilisation :

```

1 var entrepot = new Entrepot<String>();
2 entrepot.ajouterBoite(new Boite<>("Pommes", 2.5));
3 entrepot.ajouterBoite(new Boite<>("Pommes", 1.8));
4 entrepot.ajouterBoite(new Boite<>("Bananes", 3.0));
5 IO.println(entrepot);
  
```

Sortie attendue (présentée sur plusieurs lignes) :

```

[Boite{contenu=Pommes, poids=2.5},
 Boite{contenu=Pommes, poids=1.8},
 Boite{contenu=Bananes, poids=3.0}]
  
```

Pourquoi ne peut-on pas ajouter une `Boite<Integer>` à cet `Entrepot<String>` ? Vérifiez-le en ajoutant temporairement une telle instruction, puis retirez-la pour poursuivre.

Étape 3 : Regroupement par contenu et tri

Ajouter les deux méthodes suivantes dans `Entrepot<T>` :

- `regrouperParContenu()` retourne une `Map<T, List<Boite<T>>>` : les boîtes ayant des contenus égaux sont rassemblées dans la même liste ;
- `trierParPoids(List<Boite<T>> liste)` trie la liste fournie par poids croissant. Elle utilise l'ordre naturel de `Boite<T>`, avec l'appel suivant :
`liste.sort(Comparator.naturalOrder())`.

Ces méthodes sont des méthodes d'instance de `Entrepot<T>`. Le regroupement s'appuie sur sa liste interne. Le tri peut s'appliquer à une sous-liste issue du regroupement ou à toute autre liste modifiable de `Boite<T>`.

Rappel : la structure Map

Une `Map<K, V>` associe une clé de type `K` à une valeur de type `V`. Ici, une clé est un contenu et sa valeur est la liste des boîtes ayant ce contenu.

Pour un `Entrepot<String>`, on obtient par exemple :

```

Pommes -> [Boite{contenu=Pommes, poids=2.5},
           Boite{contenu=Pommes, poids=1.8}]
Bananes -> [Boite{contenu=Bananes, poids=3.0}]
  
```

Il s'agit bien d'un regroupement par **contenu** : "Pommes" et "Bananes" sont deux chaînes différentes, mais elles ont le même type `String`.

Indications pour construire le regroupement :

1. Créer une nouvelle `LinkedHashMap<T, List<Boite<T>>>`. Elle conserve l'ordre de première apparition des clés, ce qui rend les sorties ci-dessous reproductibles.
2. Pour chaque boîte, récupérer son contenu et chercher sa liste avec `get(cle)`.
3. Si la clé est absente, `get` retourne `null` : créer une nouvelle `ArrayList<Boite<T>>` et l'associer à la clé avec `put(cle, liste)`.
4. Ajouter la boîte à la liste correspondant à sa clé.

La map et les listes de groupes sont nouvelles ; elles contiennent les boîtes existantes. Trier un groupe ne doit donc pas changer l'ordre de la liste interne de l'entrepôt.

Exemple de parcours et de tri :

```

1 Map<String, List<Boite<String>>> groupes =
2   entrepot.regrouperParContenu();
3 for (String contenu : groupes.keySet()) {
4   List<Boite<String>> groupe = groupes.get(contenu);
5   entrepot.trierParPoids(groupe);
6   IO.println(contenu + " triées par poids : " + groupe);
7 }
    
```

Sortie attendue (présentée sur plusieurs lignes) :

```

Pommes triées par poids : [Boite{contenu=Pommes, poids=1.8},
                          Boite{contenu=Pommes, poids=2.5}]
Bananes triées par poids : [Boite{contenu=Bananes, poids=3.0}]
    
```

Étape 4 : Programme principal

Copiez le fichier fourni `TestEntrepot.java` dans le dossier `TP2-Entrepot`. Ce programme affiche le contenu initial, puis les boîtes regroupées par contenu, puis chaque groupe trié par poids. Son code est reproduit ci-dessous.

Exemple d'utilisation :

```

1 void main() {
2   var e = new Entrepot<String>();
3   e.ajouterBoite(new Boite<>("Pommes", 2.5));
4   e.ajouterBoite(new Boite<>("Pommes", 1.8));
5   e.ajouterBoite(new Boite<>("Bananes", 3.0));
6
7   IO.println("=== Entrepôt initial ===");
8   IO.println(e);
9   var groupes = e.regrouperParContenu();
10
11   IO.println("=== Groupes par contenu ===");
12   for (var contenu : groupes.keySet()) {
13     IO.println(contenu + " : " + groupes.get(contenu));
14   }
15
16   IO.println("=== Groupes triés par poids ===");
17   for (var contenu : groupes.keySet()) {
18     var groupe = groupes.get(contenu);
19     e.trierParPoids(groupe);
20     IO.println(contenu + " : " + groupe);
21   }
22 }
    
```

Sortie attendue (présentée sur plusieurs lignes) :

```
=== Entrepôt initial ===
[Boite{contenu=Pommes, poids=2.5},
  Boite{contenu=Pommes, poids=1.8},
  Boite{contenu=Bananes, poids=3.0}]
=== Groupes par contenu ===
Pommes : [Boite{contenu=Pommes, poids=2.5},
          Boite{contenu=Pommes, poids=1.8}]
Bananes : [Boite{contenu=Bananes, poids=3.0}]
=== Groupes triés par poids ===
Pommes : [Boite{contenu=Pommes, poids=1.8},
          Boite{contenu=Pommes, poids=2.5}]
Bananes : [Boite{contenu=Bananes, poids=3.0}]
```

Dans VS Code, enregistrez les fichiers, ouvrez `TestEntrepot.java`, puis cliquez sur **Run** au-dessus de `void main()`. Comparez le résultat affiché dans le terminal intégré avec la sortie attendue.

Vérifiez enfin le fonctionnement avec un entrepôt vide, puis avec un `Entrepot<Integer>` contenant deux boîtes de contenu 42 et une de contenu 7. Les deux boîtes contenant 42 doivent appartenir au même groupe. Aucune modification des classes génériques ne doit être nécessaire.